

Mixup Augmentation, Worked Out by Hand

Training on convex blends of examples and their labels

A complete numerical walk-through of *mixup*: the augmentation that builds brand-new training samples by linearly interpolating two real ones — *and their labels*. We take two tiny tabular rows, draw a mixing weight λ , blend both the features and the one-hot targets, and then prove the single fact that makes mixup cheap to implement: training on the soft, blended label is *identical* to a λ -weighted average of two ordinary cross-entropy losses. Nothing is hand-waved — every number below was produced by the NumPy reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It fixes two samples $x_i = [1, 2, 3, 4]$ (class 0) and $x_j = [5, 6, 7, 8]$ (class 2), picks $\lambda = 0.7$, and computes the mixed feature vector $\tilde{x}$, the mixed soft label $\tilde{y}$, and the resulting loss — twice, two different ways — to show they agree to the last decimal. It then dissects the $\text{Beta}(\alpha, \alpha)$ distribution that λ is drawn from, so you can see exactly what the knob α controls.

How to read this. Each step is self-contained and gets a coloured box isolating the idea on the cleanest possible numbers. Read this to understand what mixup *is* underneath the two-line `mixup_criterion` helper in the course, and why it acts as a strong, almost-free regulariser for limited-data tabular and time-series problems.

Concepts covered, in order: the mixup map for features $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$ · the matching label blend $\tilde{y} = \lambda y_i + (1 - \lambda)y_j$ · cross-entropy on hard vs. soft targets · the loss-equivalence identity $\lambda \text{CE}(p, y_i) + (1 - \lambda)\text{CE}(p, y_j) = \text{CE}(p, \tilde{y})$ · the $\text{Beta}(\alpha, \alpha)$ sampler and what α does · why this smooths the decision boundary · when to reach for it, and what to watch out for.

1 The setup: two samples small enough to blend by hand

Mixup operates on *pairs* of training examples. To follow it on paper we shrink to the smallest pair that still exercises every mechanism — two feature vectors and two labels — and fix their contents so each blended number is impossible to misread.

Quantity	Symbol	Value here
Sample i features	x_i	[1, 2, 3, 4]
Sample j features	x_j	[5, 6, 7, 8]
Label of i (class 0, one-hot)	y_i	[1, 0, 0]
Label of j (class 2, one-hot)	y_j	[0, 0, 1]
Number of classes	C	3
Mixing weight (this run)	λ	0.7
λ is drawn from	—	Beta(α, α)

The example. Think of two rows of a tabular dataset: one fraud-free account (class 0) and one chargeback account (class 2). Mixup does not pick one; it manufactures a *hybrid* row that is 70% the first and 30% the second, and labels it as a 70/30 mixture of the two outcomes. The model never sees this exact row in the real data — that is the point.

Intuition

Standard training shows the network a finite set of isolated points and lets it draw *any* decision boundary that separates them, including jagged ones that memorise noise. Mixup instead fills the empty space *between* examples with linearly-interpolated phantoms whose labels interpolate too. The network is thereby pushed to behave *linearly* as you walk from one example to another, which straightens and smooths the decision boundary. Smoother boundaries generalise better and produce better-calibrated probabilities — the model becomes less prone to confident nonsense in regions it never actually observed.

2 Step 1 — Blending the features: $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$

Mixup forms a **convex combination** of the two feature vectors. With $\lambda = 0.7$ the weights are $\lambda = 0.7$ on x_i and $1 - \lambda = 0.3$ on x_j , and they sum to 1 (that is what “convex” means — the result stays inside the segment joining the two points):

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j$$

Component by component:

$$\begin{aligned} \tilde{x}_0 &= 0.7 \cdot 1 + 0.3 \cdot 5 = 2.2, & \tilde{x}_1 &= 0.7 \cdot 2 + 0.3 \cdot 6 = 3.2, \\ \tilde{x}_2 &= 0.7 \cdot 3 + 0.3 \cdot 7 = 4.2, & \tilde{x}_3 &= 0.7 \cdot 4 + 0.3 \cdot 8 = 5.2. \end{aligned}$$

$$\tilde{x} = [2.2 \quad 3.2 \quad 4.2 \quad 5.2]$$

Because the two inputs differ by a constant 4 in every coordinate, the blend sits exactly $0.3 \times 4 = 1.2$ above x_i in every coordinate — a clean sanity check you can read straight off the numbers.

Mini-example — this operation in isolation

“Convex” is the whole story. Take a single feature: $x_i = 1$, $x_j = 5$. As λ sweeps $1 \rightarrow 0$ the blend $0.7 \cdot 1 + 0.3 \cdot 5 = 2.2$ traces the segment $[1, 5]$ and *never leaves it*. At $\lambda = 1$ you recover $x_i = 1$; at $\lambda = 0$ you recover $x_j = 5$; at $\lambda = 0.5$ you land on the midpoint 3. Mixup is just this straight-line interpolation applied to every feature at once.

3 Step 2 — Blending the labels: $\tilde{y} = \lambda y_i + (1 - \lambda) y_j$

The defining feature of mixup — the part beginners forget — is that the *label* is interpolated with the *same* λ . The one-hot targets $y_i = [1, 0, 0]$ and $y_j = [0, 0, 1]$ blend into a **soft label**:

$$\boxed{\tilde{y} = \lambda y_i + (1 - \lambda) y_j} = 0.7 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + 0.3 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0 \\ 0.3 \end{bmatrix}.$$

So the hybrid row carries the target $\tilde{y} = [0.7, 0, 0.3]$: “70% class 0, 0% class 1, 30% class 2.” It is still a valid probability vector — its entries are non-negative and sum to $0.7 + 0 + 0.3 = 1$ — because a convex combination of two probability vectors is again a probability vector.

Watch out

A blended feature row is *not* a hard example for class 0 that happens to look unusual; its target must be the *soft* vector $[0.7, 0, 0.3]$. If you mix the features but keep a hard label (say, just class 0), you have thrown away the entire regularising signal and are merely feeding the network corrupted inputs. Mix **both**, with the **same** λ , every time.

4 Step 3 — The loss, two ways, and why they agree

How do you train on a soft label? Recall cross-entropy against a target probability vector t over predicted class probabilities p :

$$\text{CE}(p, t) = - \sum_{c=0}^{C-1} t_c \log p_c.$$

When t is one-hot at class c^* , this collapses to the familiar $\text{CE} = -\log p_{c^*}$.

A concrete prediction. Suppose the model outputs logits $z = [0.5, -0.2, 1.3]$ on the hybrid row. Softmax turns these into probabilities:

$$p = \text{softmax}(z) = [0.269 \quad 0.133 \quad 0.598], \quad \log p = [-1.314 \quad -2.014 \quad -0.514].$$

Way 1 — the course’s `mixup_criterion`: blend two hard losses. The course computes the loss as a λ -weighted average of the cross-entropy against *each original hard label*:

$$\mathcal{L}_{\text{mix}} = \lambda \text{CE}(p, y_i) + (1 - \lambda) \text{CE}(p, y_j).$$

With $y_i = \text{class 0}$ and $y_j = \text{class 2}$:

$$\text{CE}(p, y_i) = -\log p_0 = 1.314, \quad \text{CE}(p, y_j) = -\log p_2 = 0.514,$$

$$\mathcal{L}_{\text{mix}} = 0.7(1.314) + 0.3(0.514) = 0.920 + 0.154 = \boxed{1.074}.$$

Way 2 — cross-entropy against the single soft label $\tilde{y}$.

$$\text{CE}(p, \tilde{y}) = - \sum_c \tilde{y}_c \log p_c = -(0.7(-1.314) + 0(-2.014) + 0.3(-0.514)) = \boxed{1.074}.$$

The two routes give the *same* number, 1.074. This is not a coincidence; it is an algebraic identity:

$$\lambda \text{CE}(p, y_i) + (1 - \lambda) \text{CE}(p, y_j) = \text{CE}(p, \lambda y_i + (1 - \lambda)y_j) = \text{CE}(p, \tilde{y})$$

It holds because cross-entropy is *linear in its target argument*: substituting $\tilde{y} = \lambda y_i + (1 - \lambda)y_j$ into $-\sum_c \tilde{y}_c \log p_c$ and distributing the sum reproduces the weighted average exactly.

Intuition

This identity is *why mixup is cheap to implement*. You never have to build the soft label tensor or call a soft-target loss; you keep the two original hard labels y_i, y_j and your ordinary `CrossEntropyLoss`, run it twice, and blend the two scalars with λ . Two lines of code (the course's `mixup_criterion`) deliver the full soft-label objective with zero extra machinery.

5 Step 4 — Where λ comes from: Beta(α, α)

We chose $\lambda = 0.7$ above to make the arithmetic clean. In real training, λ is *drawn fresh for every batch* from a Beta distribution with both shape parameters equal, $\lambda \sim \text{Beta}(\alpha, \alpha)$. The single hyperparameter $\alpha > 0$ controls *how much mixing* happens. Because the two shape parameters are equal, the distribution is symmetric about 0.5, so its mean is always $\frac{1}{2}$; what changes with α is the *spread*:

$$\mathbb{E}[\lambda] = \frac{1}{2}, \quad \text{Var}[\lambda] = \frac{1}{4(2\alpha + 1)}.$$

Small α gives large variance — mass piles up near 0 and 1, so most draws are *near-pristine* samples (mild mixing). Large α gives small variance — mass concentrates near 0.5, so most draws are *heavy* 50/50 blends (strong mixing). $\alpha = 1$ is the flat, uniform case.

α	Var[λ]	std[λ]	density at $\lambda=0.5$	behaviour of λ
0.2	0.179	0.423	0.319	strongly U-shaped: nearly all draws near 0 or 1
0.4	0.139	0.373	0.544	U-shaped: mild mixing (the course default)
1.0	0.083	0.289	1.000	uniform on $[0, 1]$: every blend equally likely
2.0	0.050	0.224	1.500	bell-shaped: draws cluster around 0.5
5.0	0.023	0.151	2.461	sharply peaked at 0.5: strong 50/50 mixing

The density column makes the U-shape vs. bell-shape transition concrete. For $\alpha < 1$ the density at the centre 0.5 is *below* 1 (mass fled to the edges); at $\alpha = 1$ it is exactly the flat value 1; for $\alpha > 1$ it rises above 1 as probability is pulled toward the middle.

Mini-example — this operation in isolation

At $\alpha = 0.4$ (the course default) a seeded draw of eight λ 's is

[0.895, 0.906, 0.387, 1.000, 1.000, 0.972, 0.762, 0.297].

Notice how many land close to 0 or 1: with $\lambda \approx 1$ the “mixed” sample is almost exactly x_i with almost its original label — mixup is gently perturbing real samples most of the time, and only occasionally producing an aggressive blend. The empirical mean over 100,000 draws is 0.500, matching $\mathbb{E}[\lambda] = \frac{1}{2}$ exactly.

When this is the right choice

Reach for mixup when data is **limited** and overfitting is the enemy — small tabular datasets, time-series, and vision are the classic wins. It is a remarkably *cheap, strong regulariser*: no architecture change, one extra hyperparameter (α), and a two-line loss. Start at $\alpha \approx 0.2$ – 0.4 for tabular data (mild mixing), and treat α as a dial: turn it up only if the model is still overfitting.

Watch out

Mixup blends *aligned, compatible* quantities. For time-series, mix windows that are **aligned** (same phase / length); averaging arbitrary, misaligned windows destroys temporal structure rather than regularising it. Do not naively blend **incompatible categorical** features — a 0.7/0.3 mix of one-hot category codes is rarely meaningful and is better handled in embedding space or left untouched. And a **too-large** α over-mixes: when almost every batch is a heavy 50/50 blend the targets become so soft that the model *underfits* and training loss stalls.

6 The blend, drawn

As λ runs from 1 to 0, the mixed sample slides along the straight segment from x_i to x_j , and the label slides in lockstep from $[1, 0, 0]$ to $[0, 0, 1]$. Our $\lambda = 0.7$ point sits 30% of the way across:



The indigo dots are the two real samples; the green dot is the manufactured hybrid. Every point on the line is a legal training example mixup might produce, with its label given by the same position λ along the segment. Training over the whole line — not just the two endpoints — is what straightens the decision boundary.

7 Mixup, at a glance

#	Quantity	For our example	Computed as
1	mixed features $\tilde{x}$	[2.2, 3.2, 4.2, 5.2]	$\lambda x_i + (1 - \lambda)x_j$
2	mixed label $\tilde{y}$	[0.7, 0, 0.3]	$\lambda y_i + (1 - \lambda)y_j$
3	predicted p	[0.269, 0.133, 0.598]	$\text{softmax}(z)$
4	$\text{CE}(p, y_i)$	1.314	$-\log p_0$
5	$\text{CE}(p, y_j)$	0.514	$-\log p_2$
6	blended loss	1.074	$\lambda \text{CE}_i + (1 - \lambda)\text{CE}_j$
7	soft-label loss	1.074	$\text{CE}(p, \tilde{y})$
8	$\mathbb{E}[\lambda]$	0.5	always, since shapes equal
9	$\text{Var}[\lambda]$ at $\alpha=0.4$	0.139	$1/[4(2\alpha + 1)]$

Rows 6 and 7 agreeing *is* the mixup loss identity. Print both during development as a unit test: if they ever disagree, your label blend and your loss blend are using different λ 's.

8 Equation cheat-sheet

Feature blend	$\tilde{x} = \lambda x_i + (1 - \lambda)x_j$
Label blend	$\tilde{y} = \lambda y_i + (1 - \lambda)y_j$
Mixing weight	$\lambda \sim \text{Beta}(\alpha, \alpha)$
Cross-entropy	$\text{CE}(p, t) = -\sum_c t_c \log p_c$
Mixup loss (blend two)	$\mathcal{L} = \lambda \text{CE}(p, y_i) + (1 - \lambda)\text{CE}(p, y_j)$
Mixup loss (soft label)	$\mathcal{L} = \text{CE}(p, \tilde{y})$ (equal to the above)
λ mean	$\mathbb{E}[\lambda] = \frac{1}{2}$ (equal shapes)
λ variance	$\text{Var}[\lambda] = \frac{1}{4(2\alpha + 1)}$
α dial	$\alpha < 1$ mild · $\alpha = 1$ uniform · $\alpha > 1$ strong

Appendix A Reference implementation

Every number above is reproducible from the following program (NumPy; the `mixup_criterion` blend mirrors the course's PyTorch helper exactly). Change λ or α to explore.

```
import numpy as np
np.set_printoptions(precision=3, suppress=True)

# --- the two samples and their one-hot labels ---
x_i = np.array([1., 2., 3., 4.]); y_i = np.array([1., 0., 0.]) # class 0
x_j = np.array([5., 6., 7., 8.]); y_j = np.array([0., 0., 1.]) # class 2
lam = 0.7

# --- Step 1 & 2: blend features and labels with the SAME lambda ---
x_tilde = lam * x_i + (1 - lam) * x_j # [2.2 3.2 4.2 5.2]
y_tilde = lam * y_i + (1 - lam) * y_j # [0.7 0. 0.3]
print("x_tilde =", x_tilde)
print("y_tilde =", y_tilde, " sums to", y_tilde.sum())

# --- a concrete prediction on the mixed row ---
z = np.array([0.5, -0.2, 1.3]) # logits
p = np.exp(z - z.max()); p /= p.sum() # softmax -> [0.269 0.133 0.598]
logp = np.log(p) # [-1.314 -2.014 -0.514]

# --- Step 3, Way 1: blend two hard cross-entropies (course's helper) ---
def CE(p, onehot): return -np.sum(onehot * np.log(p))
def mixup_criterion(p, y_a, y_b, lam):
    return lam * CE(p, y_a) + (1 - lam) * CE(p, y_b)
loss_blend = mixup_criterion(p, y_i, y_j, lam) # 1.074

# --- Step 3, Way 2: one cross-entropy against the soft label ---
loss_soft = -np.sum(y_tilde * logp) # 1.074

print("CE(p, y_i) = -log p[0] =", round(-logp[0], 3)) # 1.314
print("CE(p, y_j) = -log p[2] =", round(-logp[2], 3)) # 0.514
print("blended loss =", round(loss_blend, 3)) # 1.074
print("soft loss =", round(loss_soft, 3)) # 1.074
assert np.isclose(loss_blend, loss_soft) # the mixup identity

# --- Step 4: the Beta(alpha, alpha) sampler ---
for a in [0.2, 0.4, 1.0, 2.0, 5.0]:
    var = 1.0 / (4 * (2 * a + 1))
    print(f"alpha={a}: Var[lamda]={var:.3f}, std={np.sqrt(var):.3f}")

rng = np.random.default_rng(0)
draws = rng.beta(0.4, 0.4, size=8) # seeded, reproducible
print("Beta(0.4,0.4) draws:", np.round(draws, 3))
# [0.895 0.906 0.387 1. 1. 0.972 0.762 0.297]
print("mean of 100k draws:",
      round(np.random.default_rng(0).beta(0.4, 0.4, 100000).mean(), 3)) # 0.5
```

— end of document —